



GROUP ASSIGNMENT	
ISSUE DATE: 23 APRIL 2026 SUBJECT: TECHNICAL PROGRAMMING III	DUE DATE: 15 MAY 2026 CODE: TPG316C
INSTRUCTIONAL PROGRAMME: DIPLOMA IN INFORMATION TECHNOLOGY INSTRUCTIONAL PROGRAMME CODE: DP_ITC/EX_ITC	
INSTRUCTIONS: This is a group assignment, and a minimum of five (5) and a maximum of ten (10) students per group is allowed. Students are responsible for forming their own groups, and all group members must actively participate in the development and presentation of the system. - All group members' full names and student numbers must be clearly indicated in all source code files (.dart) as well as in all documentation submitted for this assignment. Failure to include this information consistently will result in penalties. - Only one member of each group should submit the assignment on behalf of the group. Multiple submissions for the same group will not be accepted. - The assignment must be submitted via Blackboard. The Blackboard submission must contain all required components as specified in these instructions. - The application must be developed using GitHub for version control. All group members are expected to contribute meaningfully, and the GitHub commit history will be reviewed to assess individual and group participation. - Each group is required to submit formal documentation of the developed system. The documentation must be submitted as a single PDF file via Blackboard. The document must consist of a minimum of five (5) pages and a maximum of ten (10) pages, and must include captioned figures of the final application screens with clear explanations of system functionality and design. The documentation must also explain how the concepts covered in Units 1 to 5 were applied and must include a clear breakdown of group member contributions. - Each group will be required to deliver a 15-minute practical presentation and live system demonstration. During the presentation, students must demonstrate the full system and explain their design and implementation decisions. The majority of marks for this assignment will be awarded during the presentation and demonstration, and not solely based on the submitted artefacts. Groups that fail to present will receive zero (0) marks for the assignment. - This assignment contributes 30% of the final semester mark for the module.	

10. **No late submissions will be accepted under any circumstances.** Students are advised to plan their work accordingly to ensure that all requirements are met before the deadline.
11. Each group must use **one project folder only**. The project folder must be named using the following format: **GROUP_<letter></letter>** (for example, GROUP_A).
12. Each group is required to submit **both a zipped version of the project folder** and a **GitHub repository link** for the project. The GitHub repository will serve as the **primary evidence of development, collaboration, and version control**, while the zipped folder will be treated as a **final backup snapshot** of the project. The zipped folder must reflect the same version of the system demonstrated during the presentation.

Reference Material: Unit 1-5

1. The reference material for this assignment is drawn from Units 1 to 5.
2. Apply concepts covered in lectures, tutorials, and practical sessions, including user interface design, state management using MVVM and Provider, routing and navigation, form handling and validation, and **Supabase** authentication and CRUD operations.

Total marks: 100

PROJECT: STUDENT ASSISTANT APPLICATION SYSTEM

Project Overview

The Information Technology Department offers **Student Assistant positions** for **first-year, second-year, and third-year modules**. Due to the growing number of applicants, the department requires a mobile application to manage Student Assistant applications in a structured and secure manner.

The system must allow authenticated students to apply for Student Assistant positions by selecting the academic level and module and indicating whether they meet the minimum requirements. Administrative staff must be able to review applications and approve or reject them accordingly.

The application must store data using **Supabase** and control access through authentication. It must be developed using **Flutter**, follow the **MVVM architecture**, make use of **Provider**, implement proper **navigation**, handle user input through **forms and validation**, and support **CRUD operations**, with development managed using **GitHub**.

1. STUDENT PORTAL SCREENS

The Student Portal must allow authenticated students to submit and manage Student Assistant applications. All student functionality must be accessible only after authentication.

1.1 Authentication Screen (Login – Read Operation)

The system must provide a single authentication mechanism for all users. Authentication must verify user identity and restrict access to system functionality based on user role.

Requirements

- Users must provide valid login credentials.
- Access to the system must be blocked until authentication succeeds.
- After authentication, the system must direct users to an appropriate interface based on their role.

Authentication must be implemented using **Supabase** Authentication.

1. 2 Home Screen (Read Operation)

*After authentication, student users must be presented with a **Home screen** that provides an overview of their application activity.*

Requirements

- View submitted Student Assistant applications.
- View the current status of each application.
- Access functionality to submit or manage applications.

Application data must be retrieved from the backend and scoped to the authenticated user.

1. 3 Student Assistant Application Form (Create / Update Operation)

The system must support the submission of a single Student Assistant application per student. The application must allow a student to apply to assist with no more than two modules.

The form must demonstrate:

- logical grouping of related fields,
- controlled input to avoid invalid selections,
- appropriate validation of user input.

Application Requirements

- The student's current year of study must be captured.
- For each module application, the academic level and module must be clearly associated.
- A second module application is optional but must be limited and validated.
- Students must confirm eligibility and submit required supporting documentation.
- Applicants must not submit more than one application.

Eligibility decisions must **not** be automated by the system and must remain the responsibility of administrative users.

1. 4 Application Detail Screen (Read / Delete Operation)

The system must allow students to view full details of a submitted application and manage it while the application status remains pending.

Requirements

- View application details.
- Edit application information while pending.
- Delete an application with confirmation.

2. SYSTEM ADMIN PORTAL (FOR ADMIN USERS ONLY)

*The system must provide an **Admin Portal** for authorised administrative users. Admin users must authenticate using the same login process as all users. After authentication, the system must determine user role and display the appropriate administrative interface.*

2.1 Admin Dashboard (Read / Update / Delete Operations)

The Admin Dashboard must support review and management of Student Assistant applications.

Requirements

- View all submitted applications.
- Review applicant information and supporting documentation.
- Approve or reject applications.
- Update application status.
- Remove invalid applications.
- Optionally filter application data for review purposes.

Administrative access must be restricted to authorised users only.

3. SUPABASE SETUP

*The application must make use of **Supabase** for authentication, database storage, and file storage. Supabase is used as the backend service for the system, and all application data must be persisted remotely.*

Backend Requirements

- Supabase Authentication must verify user identity.
- Application data must be stored persistently.
- Application records must be associated with authenticated users.
- Uploaded documents must be stored securely and linked to applications.
- Access controls must ensure data isolation between users.

For this group assignment, all group members must connect to a single Supabase project.

4. DOCUMENTATION OF THE APPLICATION

Each group is required to submit **formal documentation** of the developed system. The documentation serves as evidence of the group's understanding of the system design, functionality, and the application of concepts covered in this module.

The documentation must be submitted as **a single PDF document via Blackboard** and must consist of a **minimum of five (5) pages and a maximum of ten (10) pages**.

The document must include the following:

System Overview

- Description of the purpose of the application
- Explanation of how the system addresses the Student Assistant application process

Application Screens

- Captioned screenshots of final application screens
- Explanation of the purpose and functionality of each screen

Technical Explanation

- Explanation of MVVM architecture and state management
- Explanation of routing and navigation
- Explanation of form handling and validation
- Explanation of Supabase authentication and CRUD operations

Group Contribution

- Names and student numbers of all group members
- Clear description of each group member's contribution

Failure to submit the required documentation, or submission of incomplete documentation, may result in penalties regardless of system functionality.

5. PRESENTATION AND ASSESSMENT

Assessment prioritises **demonstration and explanation**.

Component	Marks
Presentation & System Defence	50
System Functionality	20
GitHub Collaboration	20
Documentation	10

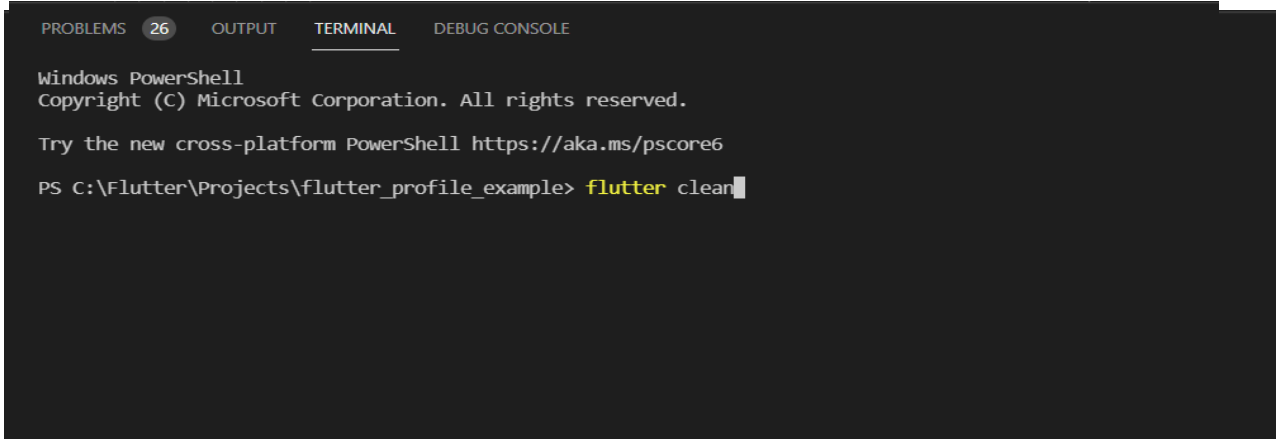
Comments

Remember to comment on your group work following the structure below:

```
/**
 * Student Numbers: 210070123, 210070456,210070789, 2100701011
 * Student Names  : John Doe, Jane Smith, Clark Kent, Bruce Lee
 * Question: Home Page
 */
```

Very Important for Submission

Before submitting your program, navigate to the terminal, and type in the following command to **decrease the file size for submission on Ethuto** as in the image below:



The screenshot shows a Windows PowerShell terminal window with the following text:

```
PROBLEMS 26 OUTPUT TERMINAL DEBUG CONSOLE

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Flutter\Projects\flutter_profile_example> flutter clear
```